

UNIVERSIDAD NACIONAL DE ENTRE RÍOS

Facultad de Ingeniería

Carrera: T.U.P.E.D

Materia: Fundamentos de Algoritmos y Estructuras de Datos

Título del Trabajo: Aplicaciones de TADs

Equipo del Docente:

Dr. Javier E. Diaz Zamboni

Bioing. Jordán F. Insfrán

Bioing. Diana Vertiz del Valle (de licencia)

Esp. Bioing. Juan Francisco Rizzato

Dr. Bioing. Feliciano Franco

Integrantes del equipo:

Franco Acevedo

Franco Tomiozzo

Comisión: 1

Fecha de entrega: 04/05/2026

Link del repositorio:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>

PROBLEMA N° 1 TAD Lista doblemente enlazada

INTRODUCCIÓN

El trabajo tiene como objetivo implementar y analizar un Tipo Abstracto de Datos (TAD) usando una Lista Doblemente Enlazada, e analizando los tiempos de función de tres métodos fundamentales: `len()`, `copiar()` e `invertir()`.

DESARROLLO

La implementación incorpora varias optimizaciones clave que mejoran el rendimiento general de la estructura:

- **Contador de tamaño:** permite que el método `__len__()` sea $O(1)$ en vez de $O(n)$.
- **Referencias a cola:** facilita la inserción y extracción en $O(1)$ al final de la lista.
- **Encapsulamiento estricto:** todos los atributos internos son privados (`__cabeza`, `__cola`, `__tamanio`).
- **Interfaz dual:** métodos clásicos (`obtener_siguiente`) y propiedades (`siguiente`) conviven para mayor compatibilidad

Método `__len__()` ($O(1)$)

El método `__len__()` retorna directamente el valor del atributo `self.__tamanio`, que se mantiene actualizado en cada operación de inserción o eliminación. Por ello, su complejidad temporal es constante.

```
def __len__(self):  
    return self.__tamanio
```

Justificación: no se recorre la lista; simplemente se devuelve el valor acumulado del contador interno, por lo que el costo es independiente del número de elementos.

Método `copiar()` ($O(n)$)

El método `copiar()` recorre completamente la lista original y genera nuevos nodos en una lista destino. El número de operaciones es proporcional a la cantidad de elementos existentes.

```
def copiar(self):
    nueva = ListaDobleEnlazada()
    actual = self.__cabeza
    while actual:
        nueva.agregar_al_final(actual.obtener_valor())
        actual = actual.obtener_siguiete()
    return nueva
```

Justificación: se visita cada nodo exactamente una vez para realizar el intercambio de punteros, lo que resulta en $O(n)$ operaciones. El intercambio final de cabeza y cola es $O(1)$.

Método copiar() ($O(n)$)

El método copiar() recorre completamente la lista original y genera nuevos nodos en una lista destino. El número de operaciones es proporcional a la cantidad de elementos existentes.

```
def invertir(self):
    actual = self.__cabeza
    while actual:
        siguiente = actual.obtener_siguiete()
        actual.asignar_siguiete(actual.obtener_anterior())
        actual.asignar_anterior(siguiente)
        actual = siguiente
    self.__cabeza, self.__cola = self.__cola, self.__cabeza
```

Método concatenar() ($O(n)$)

El método concatenar() genera una copia de la lista entrante ($O(n)$) y luego enlaza esa copia al final de la lista actual. Trabajar sobre una copia garantiza que los nodos originales de otra_lista no queden referenciados desde dos listas distintas.

```
def concatenar(self, otra_lista):
    copia = otra_lista.copiar() # no tocamos los nodos originales
    if self.esta_vacia():
        self.__cabeza = copia._ListaDobleEnlazada__cabeza
        self.__cola = copia._ListaDobleEnlazada__cola
    elif not copia.esta_vacia():
        self.__cola.asignar_siguiete(copia._ListaDobleEnlazada__cabeza)
        copia._ListaDobleEnlazada__cabeza.asignar_anterior(self.__cola)
        self.__cola = copia._ListaDobleEnlazada__cola
    self.__tamanio += len(otra_lista)
```

Justificación: El método tiene complejidad $O(n)$ debido a que la operación dominante es la llamada a copiar(), que recorre todos los nodos de otra_lista para generar una copia independiente. Las operaciones de enlace posteriores (asignar siguiente/anterior entre colas y cabezas) son $O(1)$, ya que son simples asignaciones de punteros. Por lo tanto, el tiempo de ejecución crece linealmente en función del tamaño de otra_lista, siendo la cantidad de nodos de dicha lista.

Método insertar() ($O(n)$ general / $O(1)$ en extremos)

El método insertar() acepta una posición opcional. Si no se indica, delega en agregar_al_final() ($O(1)$). Si la posición es 0 o igual al tamaño, delega en agregar_al_inicio() o agregar_al_final() respectivamente ($O(1)$). En cualquier otra posición, debe recorrer la lista hasta llegar al índice indicado ($O(n)$ en el peor caso).

```
def insertar(self, item, posicion=None):
    if posicion is None:
        self.agregar_al_final(item)
        return
    if posicion < 0 or posicion > self.__tamano:
        raise IndexError("Posición fuera de rango")
    if posicion == 0:
        self.agregar_al_inicio(item)
    elif posicion == self.__tamano:
        self.agregar_al_final(item)
    else:
        actual = self.__cabeza
        for _ in range(posicion):
            actual = actual.obtener_siguiente()
        nuevo = Nodo(item)
        anterior = actual.obtener_anterior()
        nuevo.asignar_siguiente(actual)
        nuevo.asignar_anterior(anterior)
        anterior.asignar_siguiente(nuevo)
        actual.asignar_anterior(nuevo)
        self.__tamano += 1
```

Método extraer() ($O(n)$ general / $O(1)$ en extremos)

El método extraer() elimina y retorna el elemento en la posición indicada. Al igual que insertar(), la extracción en el primer o último nodo se realiza en $O(1)$ gracias a las referencias __cabeza y __cola. Para posiciones intermedias, el recorrido es $O(n)$.

```

def extraer(self, posicion=None):
    if self.esta_vacia():
        raise IndexError("La lista está vacía")
    if posicion is None or posicion == -1:
        posicion = self.__tamano - 1
    if posicion < 0 or posicion >= self.__tamano:
        raise IndexError("Posición fuera de rango")
    if posicion == 0:
        valor = self.__cabeza.obtener_valor()
        self.__cabeza = self.__cabeza.obtener_siguiente()
        if self.__cabeza:
            self.__cabeza.asignar_anterior(None)
        else:
            self.__cola = None
    elif posicion == self.__tamano - 1:
        valor = self.__cola.obtener_valor()
        self.__cola = self.__cola.obtener_anterior()
        if self.__cola:
            self.__cola.asignar_siguiente(None)
        else:
            self.__cabeza = None
    else:
        actual = self.__cabeza
        for _ in range(posicion):
            actual = actual.obtener_siguiente()
        valor = actual.obtener_valor()
        actual.obtener_anterior().asignar_siguiente(actual.obtener_siguiente())
        actual.obtener_siguiente().asignar_anterior(actual.obtener_anterior())
    self.__tamano -= 1
    return valor

```

Métodos `__iter__()` y `__add__()` ($O(n)$)

`__iter__()` recorre la lista de cabeza a cola generando cada valor con `yield`, lo que habilita el uso de bucles `for` y comprensiones de lista sobre la estructura.

`__add__()` crea una nueva lista vacía y copia todos los elementos de ambas listas en orden, combinándolas sin modificar los originales.

```

def __iter__(self):
    actual = self.__cabeza
    while actual:
        yield actual.obtener_valor()
        actual = actual.obtener_siguiente()

def __add__(self, otra_lista):
    nueva = ListaDobleEnlazada()
    for item in self:
        nueva.agregar_al_final(item)
    for item in otra_lista:
        nueva.agregar_al_final(item)
    return nueva

```

Metodología

Los tamaños de lista evaluados fueron:

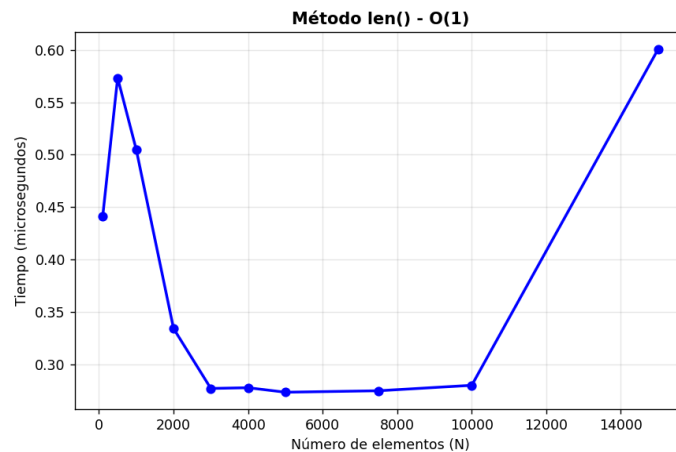
```
tamanoes = [100, 500, 1000, 2000, 3000, 4000, 5000, 7500, 10000, 15000]
```

Para cada tamaño se aplicaron las siguientes estrategias de medición:

- **__len__():** 1000 repeticiones por tamaño (operación muy rápida, requiere promedio estadístico).
- **copiar():** 10 repeticiones por tamaño (operación moderada).
- **invertir():** 10 repeticiones sobre una copia de la lista (para no alterar el estado original)

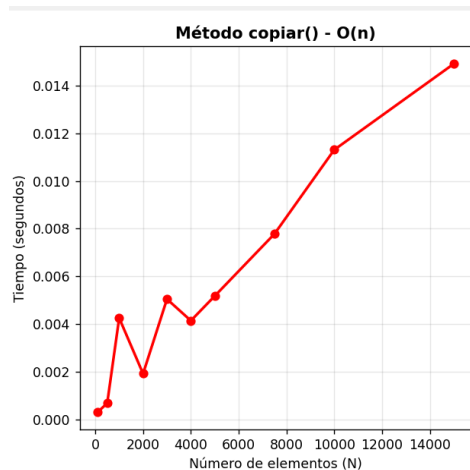
Resultados del método `__len__()` ($O(1)$)

La gráfica muestra el tiempo de ejecución de `__len__()` en microsegundos a medida que crece el número de elementos. Al no depender del tamaño de la lista, los tiempos oscilan alrededor de un valor constante ($\sim 0.28\text{--}0.60\ \mu\text{s}$). Las variaciones observadas son producto del ruido del sistema operativo (planificación de procesos, caché de CPU), no del algoritmo. Esto confirma empíricamente la complejidad $O(1)$.



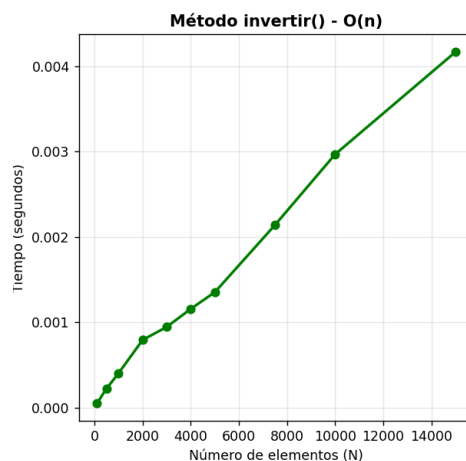
Resultados del método `copiar()` ($O(n)$)

La gráfica muestra un crecimiento aproximadamente lineal del tiempo de ejecución de `copiar()` a medida que aumenta el tamaño de la lista. Las oscilaciones en los tamaños pequeños (100–2000) se deben al ruido propio de mediciones de baja duración. A partir de $n = 3000$, la tendencia lineal se vuelve claramente visible, con tiempos que van desde $\sim 0.002\text{ s}$ hasta $\sim 0.015\text{ s}$ para $n = 15\ 000$. Esto confirma empíricamente la complejidad $O(n)$.



Resultados del método invertir() ($O(n)$)

La gráfica de invertir() muestra la curva lineal más clara de las tres: desde ~ 0.0001 s para $n = 100$ hasta ~ 0.004 s para $n = 15\,000$, con una tendencia prácticamente recta sin oscilaciones significativas. Esto se debe a que la operación de intercambio de punteros tiene un costo uniforme por nodo, sin overhead adicional. Confirma de forma contundente la complejidad $O(n)$.



CONCLUSIONES}

El análisis teórico y empírico de la Lista Doblemente Enlazada permite concluir:

- El contador `__tamano` evitar recorridos innecesarios, logrando `__len__()` en $O(1)$, confirmado por el benchmarking.
- La referencia explícita a la cola (`__cola`) permite inserciones y extracciones al final en $O(1)$.
- Los métodos `copiar()` e `invertir()` muestran crecimiento lineal tanto teórico como empírico, con la curva de `invertir()` siendo la más uniforme de las tres.

- El uso de atributos privados con doble guión bajo garantiza un encapsulamiento estricto.
- El método `concatenar()` opera sobre una copia, evitando que dos listas compartan nodos.

En conjunto, la implementación ofrece operaciones $O(1)$ en los extremos y $O(n)$ en operaciones que requieren recorrido, representando una solución equilibrada en rendimiento y claridad de código.